

Otimização de Blocos Bitcoin usando o Problema da Mochila 0/1

Relatório Técnico - Trabalho Final

Disciplina: Algoritmos e Estruturas de Dados III

Autores: Pedro D'Ávila e Stephan Heidmann

Data: Fevereiro de 2026

1. Introdução

Este trabalho apresenta uma solução prática para o problema de seleção de transações na construção de blocos Bitcoin, modelado como uma instância do Problema da Mochila 0/1 (0/1 Knapsack Problem). O objetivo é maximizar as taxas (fees) coletadas pelo minerador, respeitando o limite de tamanho do bloco.

1.1 Contexto

No protocolo Bitcoin, mineradores selecionam transações do mempool (conjunto de transações pendentes) para incluir em um novo bloco. Cada transação possui:

- **Tamanho virtual (vsize):** espaço ocupado no bloco em vbytes
- **Taxa (fee):** recompensa paga ao minerador em satoshis

O bloco possui capacidade máxima de **4.000.000 weight units**, equivalente a aproximadamente **1.000.000 vbytes**.

1.2 Analogia com a Mochila

Problema da Mochila	Construção de Bloco Bitcoin
---------------------	-----------------------------

Capacidade da mochila	Tamanho máximo do bloco
Peso do item	vsize da transação
Valor do item	fee da transação
Itens disponíveis	Transações no mempool

2. Demonstração da Intratabilidade

2.1 Definição Formal do Problema

Problema da Mochila 0/1 (Decisão):

Entrada: Conjunto de n itens, cada item i com peso w_i e valor v_i , capacidade W , e valor alvo V .

Pergunta: Existe um subconjunto $S \subseteq \{1, \dots, n\}$ tal que:

- $\sum_{i \in S} w_i \leq W$ (restrição de capacidade)
- $\sum_{i \in S} v_i \geq V$ (valor mínimo)?

2.2 Prova de NP-Completeness

O Problema da Mochila 0/1 é **NP-Completo**. A prova segue dois passos:

Passo 1: Mochila NP

Dado um certificado (subconjunto S), podemos verificar em tempo polinomial $O(n)$ se:

- A soma dos pesos não excede W
- A soma dos valores atinge V

Passo 2: Redução de Subset Sum

O problema Subset Sum (conhecido NP-Completo) reduz-se ao Knapsack em tempo polinomial:

Subset Sum: Dado conjunto $A = \{a_1, \dots, a_n\}$ e alvo T , existe $S \subseteq A$ tal que $\sum_{i \in S} a_i = T$?

Redução: Criar instância de Knapsack com:

- $w_i = v_i = a_i$ para todo i
- $W = V = T$

A resposta é SIM para Subset Sum se e somente se é SIM para Knapsack.

2.3 Implicações Práticas

Como o problema é NP-Completo:

- **Não existe algoritmo polinomial exato** (assumindo $P \neq NP$)
 - Para instâncias reais ($n \approx 100.000$ transações), soluções exatas são inviáveis
 - **Algoritmos aproximativos e heurísticos são necessários**
-

3. Soluções Propostas

Implementamos quatro abordagens algorítmicas distintas:

3.1 Algoritmo Guloso (Greedy)

Estratégia: Ordenar transações por densidade ($fee/vsize$) decrescente e selecionar enquanto couber.

Complexidade: $O(n \log n)$

Garantia: Nenhuma garantia teórica de aproximação.

3.2 Programação Dinâmica (Solução Ótima)

Estratégia: Algoritmo clássico de Bellman usando tabela de memorização.

Complexidade: $O(n \times W)$ - pseudo-polinomial

Garantia: Solução **ótima** (100% do valor máximo possível)

Limitação: Inviável para $W = 1.000.000$ devido ao consumo de memória.

3.3 Simulated Annealing (Monte Carlo)

Estratégia: Meta-heurística probabilística inspirada no recozimento de metais.

Complexidade: $O(\text{iterações} \times n)$

Garantia: Sem garantia teórica (heurístico)

Características:

- Escapa de ótimos locais através de aceitação probabilística
- Reproduzível com semente fixa
- Inicia a partir da solução gulosa

Critério de Metropolis:

3.4 FPTAS (Fully Polynomial-Time Approximation Scheme)

Estratégia: Escalar valores para reduzir espaço de busca, aplicar DP nos valores

escalados.

Complexidade: $O(n^2 / \epsilon)$

Garantia: Solução $\geq (1 - \epsilon) \times \text{OPT}$

Este é um **esquema de aproximação polinomial completo**, onde o parâmetro ϵ controla o trade-off entre qualidade e tempo:

ϵ	Garantia	Velocidade
0.01	$\geq 99\% \text{ OPT}$	Lento
0.10	$\geq 90\% \text{ OPT}$	Moderado
0.50	$\geq 50\% \text{ OPT}$	Rápido

Algoritmo:

4. Resultados Experimentais

4.1 Ambiente de Testes

- **Linguagem:** TypeScript/Node.js
- **Fonte de dados:** API mempool.space (transações reais)
- **Capacidade:** 1.000.000 vbytes (bloco Bitcoin padrão)

4.2 Comparação dos Algoritmos

Testes realizados com snapshot de 999 transações do mempool Bitcoin (capacidade: 1.000.000 vbytes):

Algoritmo	Valor (sats)	Tempo (ms)	Transações	Gap
Greedy	144.623	0.15	828	0%
SA (20k iter)	144.623	90.26	828	0%
FPTAS ($\epsilon=0.1$)	144.396	103.73	184	0.16%
DP*	32.932	6.377	219	

*DP executado com capacidade reduzida (100.000 vbytes) devido a limitação de memória.

4.3 Análise

1. **Greedy** apresentou o melhor resultado em tempo mínimo (0.15ms), demonstrando excelente relação custo-benefício para produção
2. **SA** alcançou a mesma qualidade do Greedy, validando a solução gulosa como ótimo local robusto
3. **FPTAS** ficou apenas 0.16% abaixo do melhor, respeitando a garantia teórica de $\geq 90\%$ do ótimo ($\epsilon=0.1$)
4. **DP** é inviável para capacidade real (1M vbytes), servindo apenas como referência em instâncias reduzidas

Observação: O FPTAS selecionou menos transações (184 vs 828) porque a escala de valores favorece transações com fees maiores, resultando em solução diferente mas com valor próximo.

5. Arquitetura do Sistema

5.1 Estrutura de Módulos

5.2 Qualidade do Código

- **Modularidade:** Cada algoritmo em arquivo separado
- **Testes:** 49 testes unitários cobrindo todos os algoritmos

- **Reprodutibilidade:** SA usa PRNG com semente configurável
 - **Docker:** Containerização para execução consistente
-

6. Conclusão

Este trabalho demonstrou a aplicação prática de algoritmos aproximativos e probabilísticos para resolver um problema NP-Completo real: a seleção de transações para blocos Bitcoin.

Principais Contribuições:

1. **Modelagem** do problema de construção de blocos como instância do Knapsack 0/1
2. **Implementação** de quatro abordagens algorítmicas distintas
3. **Comparação experimental** com dados reais do mempool Bitcoin
4. **Sistema completo** com CLI, testes e documentação

Trabalhos Futuros:

- Considerar dependências entre transações (CPFP)
 - Implementar algoritmos de branch-and-bound
 - Otimizar FPTAS para instâncias muito grandes
-

Referências

- [1] BELLMAN, R. Dynamic Programming. Princeton University Press, 1957.
- [2] IBARRA, O. H.; KIM, C. E. Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM*, v. 22, n. 4, p. 463-468, 1975.
- [3] KIRKPATRICK, S.; GELATT, C. D.; VECCHI, M. P. Optimization by simulated

annealing. *Science*, v. 220, n. 4598, p. 671-680, 1983.

[4] KORTE, B.; VYGEN, J. Combinatorial Optimization: Theory and Algorithms. 6th ed. Springer, 2018.

[5] NAKAMOTO, S. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008.

[6] Bitcoin Wiki. Weight Units. Disponível em: https://en.bitcoin.it/wiki/Weight_units